Orçamentista Autônomo

Guia de ferramentas e de como elas se conectam

Converge Challenge · Porto Digital, Recife · 7 e 8 de novembro de 2026

Este documento serve para explicar, sem consultar o código, qualquer peça técnica do projeto: o que é, o que faz aqui, com quem conversa, como é configurada e quais são seus limites. Foi escrito a partir do código e dos documentos do repositório. As **fatias do time** (A, B, C e D) estão explicadas na última seção.

Como ler as marcações. Onde aparece **[Fatia A]** ou **[Fatia C]**, a peça é de outra pessoa do time e **não está neste repositório** (ou está em branch ainda não juntada). Nesses casos o texto descreve o combinado, não o código. Onde aparece **[medido]**, o número vem de um teste feito no projeto.

Sumário

Sumário . . . 2

1. O produto e as camadas 4

Fluxo do usuário . . . 4

Regras que atravessam todas as ferramentas . . . 4

Onde fica a IA e onde não fica . . . 4

2. Mapa de conexões 5

3. O caminho de um orçamento, passo a passo 7

Se algo falhar . . . 8

4. Ferramentas, uma a uma 9

Python . . . 9

FastAPI e Uvicorn . . . 9

Pydantic e pydantic-settings . . . 9

httpx, requests, python-dotenv e tzdata . . . 10

Featherless AI (provedor dos modelos) . . . 10

Agente ReAct e registro de ferramentas . . . 11

Busca: Tavily e DuckDuckGo . . . 12

Extração, cálculo e lista de compra . . . 12

Cache de preços . . . 13

Supabase (Postgres e Realtime) . . . 14

Langfuse (observabilidade) . . . 14

Site: React, TypeScript e Vite . . . 15

Vercel . . . 15

Docker e docker-compose . . . 16

n8n . . . 16

Testes: pytest e geradores de arquivos . . . 16

Dados de teste sintéticos . . . 17

Scripts de verificação . . . 17

Git, GitHub e arquivos de regras para assistentes . . . 17

Google Forms (Apps Script) . . . 18

5. O registro de um item: quem escreve e quem lê 19

Campos de orcamentos . . . 19

6. Configuração por ferramenta 20

7. Perguntas que podem aparecer 21

Sobre o agente e a IA . . . 21

Sobre dados e preços . . . 21

Sobre arquitetura e operação . . . 21

Sobre negócio (ligação com o produto) . . . 22

8. O que não fazemos e o que está pendente **23**

Dizer com clareza na banca . . . 23

Pendências de integração . . . 23

9. Cola rápida **24**

10. As fatias do time **25**

Funções do produto e a fatia de cada uma . . . 25

Fatia A: Entrada e revisão (Samuel) . . . 26

Fatia B: Pesquisa agêntica de preço (Enzo) . . . 26

Fatia C: Itens esquecidos, frete e resultado (João) . . . 27

Fatia D: Negócio e apresentação . . . 28

Contratos entre as fatias . . . 28

1. O produto e as camadas

O **Orçamentista Autônomo** recebe a lista de materiais de uma obra e devolve um estudo de viabilidade: preço de cada item (faixa mínimo, médio e máximo, sempre com link da loja), quantas embalagens comprar (já com a perda), onde comprar considerando o frete pelo CEP da obra, e se o total cabe no orçamento do cliente.

Fluxo do usuário

- O arquiteto sobe o arquivo (CSV, Excel, TXT do Revit, PDF com texto) e confere os itens numa tabela editável.
- Clica em **Enviar para a IA**. O site chama o backend.
- O backend pesquisa os itens em paralelo e grava cada um no banco assim que fica pronto; o site recebe cada gravação pelo Realtime e mostra o item na tela.
- Item genérico (por exemplo, "tinta branca") vira uma **pergunta** ao arquiteto; ao responder, só aquele item é pesquisado de novo.
- No fim, o backend calcula frete, viabilidade e ganho de tempo, e o site mostra o resultado e o botão Exportar.

Regras que atravessam todas as ferramentas

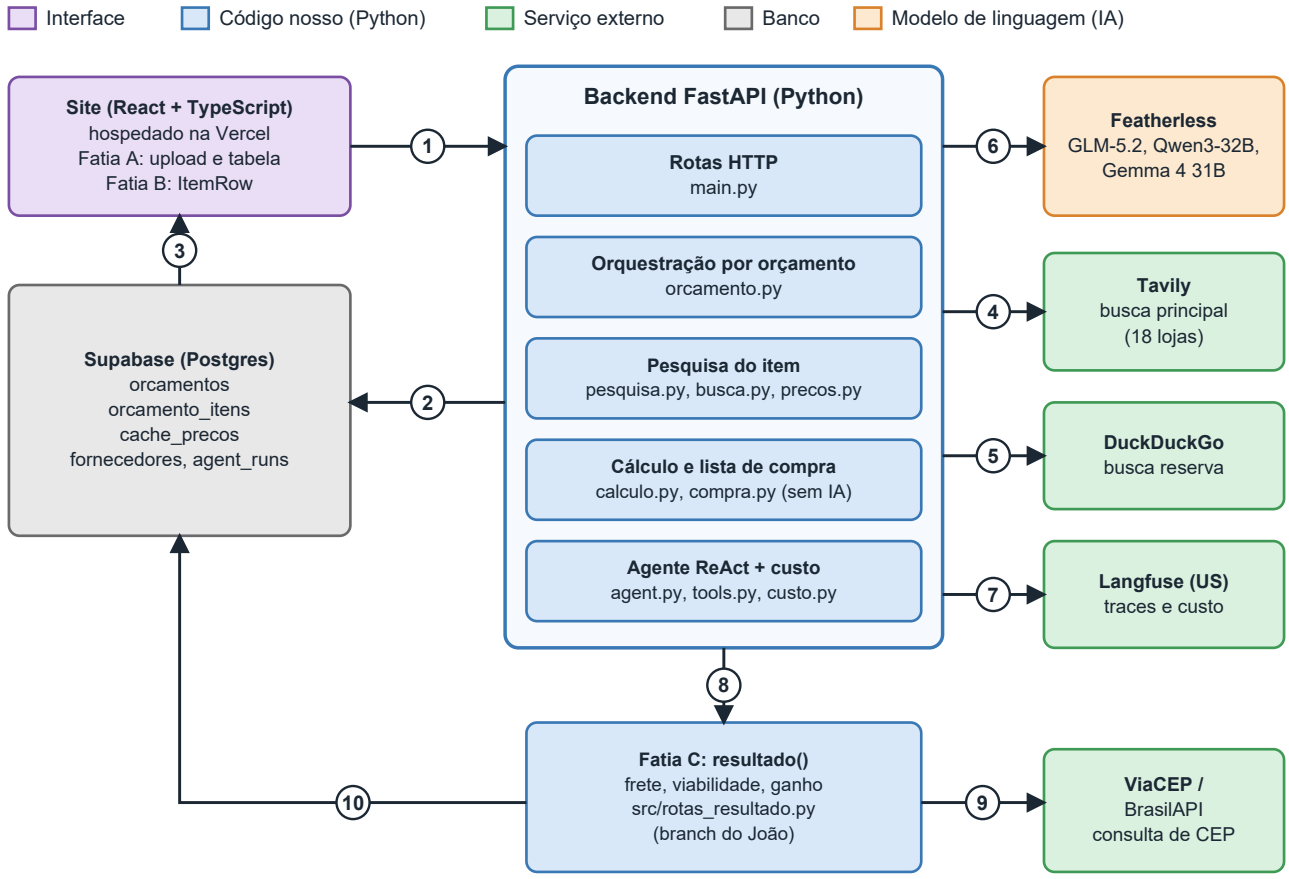
- **Nunca preço sem fonte.** Todo preço tem loja e link; o modelo de dados recusa gravar o contrário.
- **Perguntar em vez de chutar.** Falta marca, acabamento ou medida: vira pergunta, não suposição.
- **Sugerir, nunca aplicar sozinho.** Correções, complementos e similares mais baratos precisam de clique do arquiteto.
- **Conta não é feita por IA.** Mediana, embalagens, perda, frete e viabilidade são código determinístico.
- **Tudo rastreável.** Cada passo e cada chamada ao modelo aparece no Langfuse com tokens, custo e tempo.
- **Só dados sintéticos.**

Onde fica a IA e onde não fica

Etapa	Quem faz	Por quê
Ler anúncios e extrair preço, embalagem e "é o mesmo produto?"	Modelo Qwen3-32B (IA)	Texto de anúncio não tem formato fixo; é leitura de texto
Decidir como reescrever a busca quando faltam lojas	Modelo GLM-5.2 (agente ReAct)	Exige raciocinar sobre o que veio errado
Redigir a sugestão de correção na revisão [Fatia A]	Modelo (IA)	Texto livre; a regra que detecta o erro é código
Mediana, outliers, nº de embalagens, perda, confiança	Python	Conta precisa dar sempre o mesmo resultado
Frete por CEP, viabilidade, ganho [Fatia C]	Python	Idem
Parar o agente, limites, validações, regras de negócio	Python	Quem decide parar é o código, não o modelo

2. Mapa de conexões

O diagrama mostra as peças e as conexões numeradas; a tabela abaixo diz, para cada número, o protocolo, o que trafega e onde está no código.

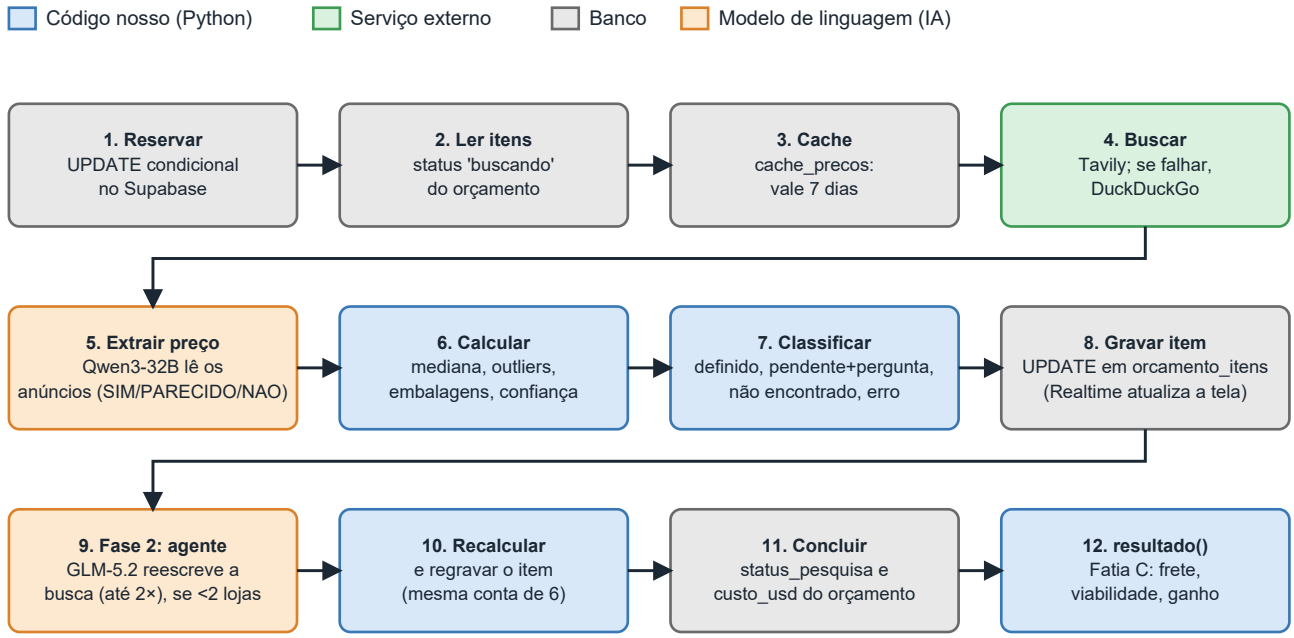


Nº	De → Para	Como (protocolo / biblioteca)	O que trafega	Código
1	Site → Backend	HTTPS, JSON (FastAPI)	Contrato do orçamento (obra, CEP, itens revisados); pedido de repesquisa de um item [Fatia A]	main.py
2	Backend → Supabase	HTTPS (PostgREST) pelo supabase-py, com a chave secreta service_role	Lê itens com status 'buscando'; grava o resultado de cada item; status, tempos e custo do orçamento; lê e grava o cache	orcamento.py, cache.py, database.py
3	Supabase → Site	WebSocket (Supabase Realtime), com a chave pública anon	Cada UPDATE em orcamento_itens e orcamentos chega à tela sem o site perguntar	[Fatia A] useOrcamento
4	Backend → Tavily	HTTPS pelo SDK tavily-python (AsyncTavilyClient)	Consulta de busca; volta lista de resultados com texto e imagens, só das 18 lojas permitidas	busca.py
5	Backend → DuckDuckGo	Biblioteca ddgs (sem chave)	Consulta; volta até 25 resultados (região br-pt). Só se o Tavily falhar	busca.py
6	Backend → Featherless	HTTPS, SDK openai apontando para api.featherless.ai/v1	Prompts e respostas dos modelos; volta o texto, as chamadas de ferramenta e usage.cost	precos.py, agent.py, pesquisa.py
7	Backend → Langfuse	SDK Langfuse (OpenTelemetry), envio assíncrono, chaves pk-lf e sk-lf	Árvore de passos, entradas e saídas, tokens e custo por chamada	decoradores @observe

Nº	De → Para	Como (protocolo / biblioteca)	O que trafega	Código
8	Orquestração → Fatia C	Chamada de função Python no mesmo processo	O id do orçamento; a função lê o banco e grava o resultado	orcamento.py (GANCHOS_FIM)
9	Fatia C → ViaCEP / BrasilAPI	HTTPS REST, gratuito e sem chave	CEP da obra; volta endereço e coordenadas para a distância às lojas	[Fatia C]
10	Fatia C → Supabase	mesmo canal do nº 2	Lê fornecedores e itens; grava logística, viabilidade, ganho em orcamentos	[Fatia C]
—	Backend → sites das lojas (não desenhada)	HTTPS pelo httpx (timeout de 4 s)	Confere se a URL da imagem escolhida realmente carrega	busca.py

Por que o Realtime é a ligação mais importante da demo. O site não pergunta "já terminou?". O backend grava cada item no banco assim que ele fica pronto e o banco avisa o site. Por isso os itens aparecem um a um, e por isso o backend precisa gravar item por item em vez de devolver tudo no final.

3. O caminho de um orçamento, passo a passo



Passo	O que acontece	Ferramenta	Detalhe que costuma ser perguntado
1. Reservar	Um UPDATE condicional marca o orçamento como 'pesquisando'. Só passa se estiver 'pendente' ou 'falhou', ou se estiver 'pesquisando' há mais de 3 minutos (execução morta pela Vercel).	Supabase	Evita duas pesquisas do mesmo orçamento se o usuário clicar duas vezes. É uma única instrução, então é atômica.
2. Ler itens	Busca os itens do orçamento com status 'buscando'.	Supabase	Itens 'pendente_especificacao' sem resposta não são relidos.
3. Cache	Procura o produto pela descrição normalizada. Válido por 7 dias; vencido só serve de reserva.	Supabase (cache_precos)	Guarda as ofertas conferidas e a imagem, não a conta, porque a conta depende da quantidade de cada orçamento.
4. Buscar	Monta a consulta e busca só nas 18 lojas da lista fixa; escolhe uma imagem do produto.	Tavily; DuckDuckGo se falhar	Até 8 buscas simultâneas. A imagem é pontuada por cor e medida citadas no título e testada com httpx.
5. Extrair	O modelo lê os anúncios achados e devolve por linha: índice, preço, embalagem, SIM/PARECIDO/NAO.	Featherless (Qwen3-32B; reserva Gemma)	O preço só vale se aparecer escrito no texto do anúncio. Resposta fora do formato é descartada.
6. Calcular	Converte tudo para preço por unidade base, remove outliers, calcula mín, mediana, máx, escolhe a embalagem de referência, calcula embalagens com perda e a confiança.	Python	Sem IA. O mesmo item dá sempre o mesmo resultado.
7. Classificar	Define o status: definido, pendente (faixa estimada + pergunta), não encontrado ou erro.	Python	Só produtos "parecidos" com preço viram estimativa com pergunta; nada é afirmado como preço exato.
8. Gravar	UPDATE do item em orcamento_itens.	Supabase → Realtime → Site	Se a gravação de um item falhar, os outros continuam.
9. Fase 2	Para itens com menos de 2 lojas, o agente reescreve a consulta e busca de novo, no máximo 2 vezes.	Featherless (GLM-5.2) + ferramenta buscar_preco	Só roda depois da fase 1 de todos os itens, para não atrasar os itens fáceis. Prazo total de 60 s.

Passo	O que acontece	Ferramenta	Detalhe que costuma ser perguntado
10. Recalcular	Junta as ofertas novas às antigas, repete o cálculo e regrava.	Python, Supabase	Item que não coube no prazo fica com o resultado da fase 1.
11. Concluir	Marca a pesquisa como 'concluida' e grava o custo em USD somado das chamadas ao modelo.	Supabase	O custo vem do campo usage.cost devolvido pela Featherless.
12. resultado()	A Fatia C calcula frete, viabilidade e ganho e grava em orcamentos.	[Fatia C]	Se o módulo ainda não existir, o gancho é pulado e vai para o log. Roda de novo após cada repesquisa.

Se algo falhar

Falha	O que o sistema faz
Tavily fora do ar ou sem cota	Usa o DuckDuckGo. Com várias chaves em TAVILY_API_KEYS, usa as outras em rodízio
Modelo de extração falha (HTTP 400/503, timeout, resposta sem linhas)	Tenta o modelo reserva; se também falhar, o item segue sem ofertas novas
Resposta vazia do modelo	Uma nova tentativa depois de uma pausa curta
Agente passa de 40 s ou dá erro	O item segue com o que a primeira busca achou
Nenhuma loja com preço	Cache vencido com a data ("preço de dd/mm"); sem nada, status "não encontrado" e botão Pesquisar de novo
Erro ao gravar um item	É registrado no log; os demais itens não são afetados
Erro geral (banco fora do ar, por exemplo)	O orçamento vira 'falhou', o que libera um novo clique; o erro fica em pesquisa_erro
Fatia C falha	A pesquisa continua válida; o erro vai para o log
Langfuse sem chaves ou desligado	O tracing desliga sozinho e o resto funciona igual

4. Ferramentas, uma a uma

Python

O que é	Linguagem de programação em que todo o backend está escrito.
Papel aqui	Agente, pesquisa de preço, cálculos, acesso ao banco e rotas HTTP. Versão 3.11 ou superior (a imagem Docker usa 3.12).
Conecta com	Todas as bibliotecas e serviços listados neste guia.
Código	src/ (backend), scripts/ (utilitários), tests/ (testes)

P: Por que Python?
As bibliotecas de modelos de linguagem, busca, banco e validação que usamos têm SDK em Python, e é a linguagem que o time dominava.

P: Por que o site não é em Python também?
O site é feito em React e TypeScript, que rodam no navegador. Ele conversa com o backend por HTTP e com o banco pelo Realtime.

P: O que é async e por que o código usa?
Async permite que o programa espere uma resposta de rede (busca, modelo, banco) sem ficar parado, então outros itens são pesquisados nesse tempo. É o que permite pesquisar vários itens em paralelo no mesmo processo.

FastAPI e Uvicorn

O que são	FastAPI é um framework para criar APIs HTTP em Python. Uvicorn é o servidor que executa o FastAPI.
Papel aqui	Expõe as rotas: GET /health (mostra se LLM, Supabase e Langfuse estão configurados); POST /agent/run (agente genérico); POST /orcamento-itens/{id}/repesquisar (botões Pesquisar de novo e Responder); /docs (página Swagger automática). POST /orcamentos e POST /orcamentos/{id}/resultado ficam nas Fatias A e C.
Conecta com	Site (conexão 1), Supabase, Featherless, Langfuse. O servidor envia os traces pendentes ao Langfuse quando desliga.
Código	src/main.py. Rodar local: uvicorn src.main:app --reload (porta 8000)

P: Por que FastAPI?
Valida a entrada pelos modelos Pydantic, gera a documentação em /docs (os jurados podem testar a API ali) e suporta async.

P: Qual a diferença entre FastAPI e Uvicorn?
FastAPI define o que cada rota faz. Uvicorn recebe as conexões da rede e entrega as requisições ao FastAPI.

P: O que a rota /health mostra?
O ambiente, se o LLM, o Supabase e o Langfuse estão configurados, o modelo em uso e a lista de ferramentas do agente. Serve para conferir rápido se o deploy está completo.

P: A API é protegida?
Não. O CORS está aberto para qualquer origem e as rotas não exigem autenticação. É uma decisão consciente para a demo; em produção seria preciso restringir a origem e autenticar.

Pydantic e pydantic-settings

O que são	Pydantic valida dados por tipos. pydantic-settings lê variáveis de ambiente e o arquivo .env e as transforma em configuração tipada.
Papel aqui	src/modelos.py define o contrato do item (ItemEntrada, Fonte, ResultadoItem). src/tools.py gera o esquema JSON das ferramentas do agente a partir dos modelos. src/config.py concentra todas as chaves e limites.
Conecta com	Todo o código; o esquema gerado é o que o modelo de linguagem recebe como descrição das ferramentas.

Código	src/modelos.py, src/tools.py, src/config.py

P: Como vocês garantem que a IA não inventa preço?
Três travas em camadas: (1) o preço que o modelo devolve só é aceito se aparecer escrito no texto do anúncio (src/precos.py); (2) o ResultadoItem recusa preço sem fonte, item 'definido' sem preço, pendente sem pergunta e faixa com mínimo maior que o máximo; (3) a conta final é Python, não modelo.

P: O que acontece se o modelo inventar um argumento para uma ferramenta?
As ferramentas usam extra=forbid: o argumento inventado é recusado e o erro volta ao modelo como observação, para ele corrigir.

P: Por que a configuração fica toda em config.py?
Chaves e limites vêm só do .env, então mudar um limite não exige mexer no código, e nenhuma chave fica escrita nos arquivos versionados.

httpx, requests, python-dotenv e tzdata

O que são	httpx e requests são clientes HTTP (httpx suporta async). python-dotenv carrega arquivos .env. tzdata fornece a base de fusos horários no Windows.
Papel aqui	httpx confere se a URL da imagem do produto realmente carrega (timeout de 4 s). tzdata permite ao agente genérico informar a hora de Recife (zoneinfo).
Observação	requests e python-dotenv estão em requirements.txt, mas o código do backend não os importa diretamente: a leitura do .env é feita pelo pydantic-settings.
Código	src/busca.py (httpx), src/tools.py (zoneinfo)

P: Para que serve conferir a imagem?
Link de imagem de anúncio pode estar quebrado ou bloqueado. Só uma imagem que carrega é aceita; senão o item fica sem foto em vez de mostrar um ícone quebrado.

P: Por que o tzdata está nas dependências?
O Windows não traz a base de fusos horários que o Python usa; sem ela a ferramenta de hora em Recife falharia.

Featherless AI (provedor dos modelos)

O que é	Serviço que oferece modelos abertos por uma API compatível com a da OpenAI.
Papel aqui	Único provedor de modelos de linguagem. Usamos o SDK openai com base_url https://api.featherless.ai/v1.
Conecta com	Backend (conexão 6). Cada chamada devolve usage (tokens e cost), que alimenta o Langfuse e o custo do orçamento.
Configuração	FEATHERLESS_API_KEY, FEATHERLESS_BASE_URL, FEATHERLESS_MODEL, FEATHERLESS_MODEL_FAST, FEATHERLESS_MODEL_FALLBACK, LLM_TEMPERATURE (0,2), LLM_TIMEOUT_SECONDS (60), LLM_MAX_RETRIES (3).
Código	src/agent.py (cliente e loop), src/precos.py (extração), src/pesquisa.py (agente da pesquisa)

P: Por que Featherless e não outro provedor?
É o gateway do hackathon (cada integrante tem crédito) e permite trocar o modelo apenas mudando o nome no .env.

P: O que acontece se o modelo principal falhar?
Nos erros HTTP 400 (modelo sem suporte ou frio) e 503 (sem capacidade), o agente tenta o modelo reserva (FEATHERLESS_MODEL_FALLBACK). A extração tem o seu próprio reserva (Gemma).

P: Quanto custa?
O custo é medido, não estimado: soma-se o usage.cost de cada chamada por orçamento (src/custo.py) e grava-se em orcamentos.custo_usd, que a tela mostra como "Custo de IA". Citar só o valor medido.

Limites: há limite de chamadas simultâneas na conta [medido]: 6 chamadas ao Qwen3-32B ao mesmo tempo geraram 3 respostas vazias. Por isso existem os limites de concorrência da seção 4.5.

Os três modelos

Modelo	Função	Por que este
zai-org/GLM-5.2	Agente da pesquisa e agente genérico (tool calling)	Faz tool calling nativo (testado). É o mais lento porque raciocina antes de responder, então só roda nos itens difíceis
Qwen/Qwen3-32B	Extrair preço e conferir se é o mesmo produto	Rápido (≈5 s) e acertou 5 de 5 na avaliação da etapa 3 [medido]
google/gemma-4-31B-it	Reserva da extração	Também acertou 5 de 5 na mesma avaliação [medido]

P: Por que usar três modelos?

Extrair preço acontece em todos os itens e é uma tarefa simples, então usa o modelo rápido. O raciocínio mais caro (reescrever a busca) só acontece nos itens em que a primeira busca falhou.

P: O que é tool calling?

O modelo responde, em formato estruturado, qual função quer chamar e com quais argumentos. Nosso código executa a função e devolve o resultado ao modelo como nova mensagem.

P: O GLM-5.2 "pensar" causa problema?

Sim, se o limite de tokens for baixo: ele gasta o limite pensando e a resposta final vem vazia. Por isso não limitamos tokens e há uma nova tentativa quando a resposta vem vazia.

Agente ReAct e registro de ferramentas

O que é	ReAct é um padrão em que o modelo repete três passos: raciocina (decide o que fazer), age (chama uma ferramenta) e observa (lê o resultado), até dar a resposta final.
Papel aqui	Dois agentes. O genérico (agent.py) tem limite de passos e escalonamento humano. O pesquisador de preço (pesquisa.py) tem uma ferramenta, buscar_preco(consulta), e só entra quando a primeira busca achou menos de 2 lojas com o mesmo produto.
Conecta com	Featherless (raciocínio), Tavily/DuckDuckGo (pela ferramenta buscar_preco), Langfuse (cada passo é um span).
Configuração	AGENT_MAX_STEPS (8), PESQUISA_MIN_LOJAS (2), PESQUISA_MAX_REFAZER (2), PESQUISA_TIMEOUT_AGENTE_S (40), PESQUISA_CONCORRENCIA_AGENTE (1), PESQUISA_AGENTES_SIMULTANEOS (2), PESQUISA_PRAZO_AGENTES_S (60).
Código	src/agent.py, src/tools.py, src/pesquisa.py, prompts/pesquisador_precos.md, prompts/system_prompt.md

P: Isso é mesmo um agente autônomo?

Sim: decide sozinho se precisa refazer a busca, como reescrever a consulta (inclui o código do modelo, simplifica ou explicita a medida, nunca troca marca nem medida) e quando parar. As regras de negócio ficam no código ao redor.

P: Quem decide que o agente terminou?

O código. Ele encerra quando já há lojas suficientes ou quando as buscas extras acabaram, sem gastar uma chamada só para o modelo dizer "fim" [medido: cerca de 10 s a menos por item difícil].

P: E se o agente errar ou travar?

Argumento inválido ou ferramenta com erro volta ao modelo como observação para ele corrigir. No agente genérico, 3 falhas seguidas escalam para um humano (escalate_to_human). No pesquisador, qualquer falha deixa o item com o resultado da primeira busca.

P: O agente toma decisões sobre pessoas?

Não. Regra do repositório: decisões sobre pessoas vão sempre para escalate_to_human. O produto atual é de orçamento de obra.

P: O que significa status 'escalated'?

O agente chamou escalate_to_human ou acumulou 3 falhas consecutivas de ferramenta. Os outros status finais são completed, max_steps_reached e error.

Limites: o pesquisador faz no máximo 2 buscas extras por item e 40 s por item; 1 chamada ao GLM por vez; 2 itens com agente ao mesmo tempo; 60 s de prazo para toda a fase 2.

Busca: Tavily e DuckDuckGo

O que são	Tavily: API de busca na web para uso por modelos de linguagem, que devolve texto e imagens. DuckDuckGo: busca usada pela biblioteca ddgs, sem chave.
Papel aqui	Tavily é a busca principal, restrita por include_domains a 18 lojas: leroymerlin.com.br, cec.com.br, telhanorte.com.br, obramax.com.br, ferreiracosta.com, madeiramadeira.com.br, amazon.com.br, mercadolivre.com.br, magazineluiza.com.br, casasbahia.com.br, americanas.com.br, dicico.com.br, loja.suvinil.com.br, lojadexco.com.br, tramontina.com.br, sertao.com.br, padovani.com.br, tumelero.com.br. DuckDuckGo é a reserva.
Conecta com	Backend (conexões 4 e 5). A saída alimenta a extração (Qwen3-32B). O campo dominio de cada fonte é a chave que a Fatia C usa para casar com a tabela fornecedores.
Configuração	TAVILY_API_KEYS (várias chaves separadas por vírgula, em rodízio), PESQUISA_CONCORRENCIA_BUSCA (8), PESQUISA_MAX_LOJAS (5).
Código	src/busca.py (consulta, busca, imagem), prompts/pesquisador_precos.md

P: De onde vêm os preços?

De anúncios nas lojas da lista fixa, encontrados pelo Tavily. Não temos acordo comercial nem API própria das lojas.

P: Por que uma lista fixa de lojas?

Evita blog, fórum, preço em dólar e marketplace desconhecido. O custo é não ver lojas fora da lista; a lista pode ser ampliada em LOJAS (busca.py).

P: Os preços são em tempo real?

São do momento da busca. Itens repetidos usam o cache por até 7 dias; o cache vencido só entra como reserva, com a data mostrada.

P: Como escolhem a imagem do produto?

Cada imagem recebe uma nota pelo que o título e a URL citam (cor e medidas do produto pedido), e só é aceita se a URL realmente carregar (checagem com httpx, 4 s). O cache guarda a imagem escolhida, então o mesmo produto sempre mostra a mesma foto.

P: Por que Tavily e DuckDuckGo juntos?

Redundância: se o Tavily falhar ou a cota acabar, a pesquisa continua com o DuckDuckGo.

Limites: preço de anúncio pode estar desatualizado ou mudar no carrinho. Por isso a tela mostra faixa, confiança e o link para conferir. Detalhe técnico [medido]: usar country='brazil' junto com include_domains faz o Tavily devolver 0 resultados, então o parâmetro não é usado.

Extração, cálculo e lista de compra

Extração (precos.py)	O modelo lê os anúncios e devolve uma linha por anúncio: índice | preço | embalagem | SIM, PARECIDO ou NAO. Ignora parcela, frete e desconto só no Pix. Kit usa o preço do kit. Palavras inteiras (e não letras soltas) no prompt, porque com letras soltas os modelos copiavam o nome do campo [medido]. Só ofertas SIM com preço entram na média.
Cálculo (calculo.py)	1) converte cada oferta para preço por unidade base (R$/m², R$/L, R$/kg, R$/un, R$/m); 2) remove outliers fora de 40% a 250% da mediana; 3) calcula mínimo, mediana e máximo; 4) escolhe a embalagem de referência (a de menor custo total para a quantidade pedida); 5) converte a faixa para R$ por embalagem; 6) calcula a confiança.
Confiança	alta: 3 ou mais lojas e dispersão até 25%. média: 2 ou mais lojas e dispersão até 50%. baixa: o restante.
Lista de compra (compra.py)	Entende embalagens ("lata 18 L", "caixa 2,16 m²", "kit 2 latas 18 L"), aplica perda por tipo de material e arredonda o número de embalagens para cima. Rendimento de tinta (m² por litro, já com 2 demãos) e consumo de argamassa e rejunte (kg por m²) são premissas; a tela mostra "≈" quando elas são usadas.
Concorrência	asyncio com limites: 8 buscas simultâneas, 3 chamadas de extração, 1 chamada do agente.

P: Por que mediana e não média?

Um anúncio com preço muito fora distorce a média; a mediana quase não muda. Além disso, outliers são removidos antes.

P: Como convertem "180 m² de tinta" em latas?
Pelo rendimento por litro (premissa por tipo de produto, com 2 demãos), mais a perda, dividido pelo conteúdo da embalagem escolhida e arredondado para cima.

P: Por que o cálculo não usa IA?
Para o mesmo item dar sempre o mesmo resultado e para o número poder ser auditado. O modelo só lê texto de anúncio.

P: Como o sistema trata item genérico?
Item sem marca, linha ou medida recebe faixa estimada (campo estimativa = verdadeiro) e uma pergunta objetiva ao arquiteto. Ao responder, o item é pesquisado de novo, ignorando o cache.

P: O que significa cada status?
buscando (na fila), definido (tem preço com fonte), pendente_especificacao (falta informação; tem pergunta), nao_encontrado (sem loja com preço; tem motivo) e erro (falha interna; tem motivo).

P: Por que limitar a concorrência?
Sem limite o provedor devolvia respostas vazias, e o agente esperava vaga até estourar o tempo [medido na etapa 9: o primeiro item chegava em 33 s e 12 agentes ficaram sem vaga]. Hoje o agente tem cota própria e roda depois da fase rápida.

Cache de preços

O que é	Tabela cache_precos no Supabase com o resultado da busca de produtos já pesquisados.
O que guarda	As ofertas conferidas (loja, domínio, url, título, preço, embalagem, se é o mesmo produto), a imagem escolhida, o modelo que extraiu e o provedor da busca. **Não guarda a conta final**, porque ela depende da quantidade de cada orçamento e é refeita na hora.
Chave	Descrição normalizada: minúsculas, sem acento, unidades coladas ao número (18 L vira 18l), sem palavras vazias, sem repetição e em ordem alfabética. "Tinta Suvinil Toque de Seda, Branco Neve 18 L" e "tinta suvinil branco neve toque seda 18L" caem na mesma chave.
Validade	7 dias (PESQUISA_CACHE_DIAS). Cache vencido continua servindo de reserva quando a busca falha, com a data mostrada ("preço de dd/mm").
Conecta com	Supabase (leitura e gravação pelo backend com a chave service_role); a pesquisa lê antes de buscar e grava depois de conferir.
Código	src/cache.py, sql/002_cache_precos.sql, scripts/aquecer_cache.py

P: Por que guardar as ofertas e não o preço final?
O número de embalagens e a escolha da embalagem de referência dependem da quantidade do orçamento. Guardando as ofertas, o mesmo produto serve a orçamentos de tamanhos diferentes e a conta é refeita instantaneamente.

P: O cache não mostra preço velho?
Vale por 7 dias. Depois disso o preço só entra se a busca falhar, e a tela mostra a data. Um repesquisar (botão ou resposta do arquiteto) ignora o cache.

P: E se a tabela não existir ou o banco falhar?
O cache nunca derruba a pesquisa: tabela ausente, rede fora ou dado estranho contam como "não está no cache".

P: Para que serve o aquecer_cache na demo?
Pesquisa todos os itens do arquivo da demo e deixa no cache a fração mais rápida (padrão: metade). Na demo, metade aparece na hora e metade chega ao vivo, sem ninguém esperar um item lento.

Supabase (Postgres e Realtime)

O que é	Postgres gerenciado, com API REST, autenticação e Realtime (o front recebe mudanças de tabelas por WebSocket).
Papel aqui	Banco de tudo e canal de tempo real entre backend e site.
Tabelas	orcamentos (um por upload: status da pesquisa, tempos, erro, custo, logística, viabilidade, ganho); orcamento_itens (um por item; **o site escuta esta tabela**); cache_precos (sql/002); fornecedores (lojas de Recife) [Fatia C]; agent_runs (auditoria do agente genérico, sql/001).
Chaves	Backend usa a chave service_role (ignora RLS) e por isso só pode ficar no servidor. O site usa a chave anon, com RLS liberando só a leitura das tabelas de que a tela precisa. cache_precos tem RLS ligado sem política para o site.
Configuração	SUPABASE_URL, SUPABASE_KEY (service_role). Na Vercel, as variáveis não levam aspas.
Código	src/database.py (cliente), src/orcamento.py (BancoSupabase), src/cache.py, sql/

P: Por que Realtime em vez de o site consultar de tempos em tempos?

O backend grava cada item quando fica pronto e o banco avisa o site na hora. A tela mostra os itens um a um sem o site precisar perguntar.

P: Como evitam pesquisar o mesmo orçamento duas vezes?

reservar() é um único UPDATE condicional (atômico): só altera se o status for 'pendente' ou 'falhou', ou 'pesquisando' há mais de 3 minutos. Se nenhuma linha for alterada, a segunda requisição responde que já está em andamento.

P: Por que a chave service_role é um risco?

Ela ignora as regras de acesso por linha. Por isso fica só no backend e nunca vai para o site nem para o Git.

P: Quem define as tabelas orcamentos e orcamento_itens?

A Fatia A (Samuel), em um arquivo SQL único, juntando os campos que B e C pedirem. Neste repositório só estão sql/001, 002 e 003. A seção 5 lista as colunas que a Fatia B lê e grava.

Limites: colunas que a Fatia B grava e que dependem do SQL do Samuel: premissa (em orcamento_itens) e custo_usd (em orcamentos). Sem elas, o custo some da tela e a gravação de premissa pode falhar.

Langfuse (observabilidade)

O que é	Plataforma de observabilidade para aplicações com modelos de linguagem: registra cada execução como um trace (árvore de passos) com entradas, saídas, tokens, custo e tempo.
Papel aqui	Mostra de onde saiu cada preço e o que o agente fez. @observe em cada etapa (orçamento, pesquisa de itens, busca, extração, agente, chamada ao modelo). Uma árvore por orçamento (session_id = id do orçamento) e um galho por item, com as iterações "errou e refez" visíveis.
Conecta com	Backend (conexão 7). O SDK usa OpenTelemetry e envia de forma assíncrona; o servidor força o envio dos traces pendentes ao desligar. Cada chamada ao modelo registra usage_details (tokens) e cost_details (o custo vem da Featherless).
Configuração	LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY, LANGFUSE_BASE_URL=https://us.cloud.langfuse.com (projeto na região US; este é o nome atual e tem prioridade sobre LANGFUSE_HOST), LANGFUSE_ENABLED. get_settings() exporta as chaves para o ambiente do processo, porque o SDK lê do ambiente e não do .env.
Código	decoradores @observe em agent.py, pesquisa.py, busca.py, precos.py, cache.py, orcamento.py; config.py

P: Para que serve na demo?

Abrir o trace de um item em que o agente errou e refez a busca: dá para ver a consulta usada, o que as lojas devolveram, o raciocínio e o custo daquela etapa.

P: O que acontece sem as chaves?

Se as chaves forem ausentes ou placeholders, o tracing desliga sozinho (LANGFUSE_TRACING_ENABLED=false) e a API continua funcionando, só sem traces.

P: Que dados ficam no Langfuse?

Textos dos itens, resultados de busca e respostas dos modelos. Como usamos apenas dados sintéticos, não há dado real de cliente.

P: Qual a diferença entre Langfuse e a tabela agent_runs?

agent_runs guarda a trilha de auditoria do agente genérico no nosso banco (com o link do trace). O Langfuse mostra a árvore detalhada de cada execução, incluindo tokens e custo por chamada.

Site: React, TypeScript e Vite

O que são	React monta a interface em componentes; TypeScript acrescenta tipos; Vite é o servidor de desenvolvimento e empacotador.
No repositório	ui-fatia-b/: ItemRow (imagem, produto, "180 m² → 2 latas de 18 L", cota de preço, confiança, links, pergunta), PriceDimension (a cota mín–médio–máx), StatusBadge e uma prévia (npm run dev, porta 5174) que simula o Realtime com dados em exemplo-real.json, para validar a tela antes de o banco existir.
Fora do repositório	O site final é da Fatia A (Samuel): framework (o padrão da Vercel é Next.js), leitura de arquivo no navegador (PapaParse para CSV, SheetJS para Excel), tabela editável, revisor de erros e o hook useOrcamento(id) com o Realtime.
Linguagem visual	"Prancha técnica": o preço aparece como cota; o carregamento é uma cota tracejada com hachura a 45° até o dado chegar. Textos em português. Guia em skills/04-ui-design.md.

P: Os jurados veem de onde veio o preço?

Cada item mostra as fontes (loja, link e preço da embalagem), a confiança e a premissa da conta, quando existe.

P: Como a tela sabe que um item ficou pronto?

Escuta orcamento_itens pelo Realtime; cada UPDATE do backend troca o status do item de 'buscando' para o resultado.

P: O que é a cota de preço?

A forma de mostrar a faixa de preço de um item: os três valores (mínimo, médio e máximo) alinhados numa régua, como uma cota de desenho técnico, em vez de um número único que pareceria mais certo do que é.

P: A prévia depende do backend?

Não. A prévia em ui-fatia-b simula o Realtime com um JSON de exemplo, para validar a tela antes de o banco e o backend estarem prontos.

P: Por que React, TypeScript e Vite?

React divide a tela em componentes reutilizáveis (como ItemRow); TypeScript detecta erro de tipo antes de rodar; Vite inicia o servidor de desenvolvimento rapidamente.

Limites: pontos de atenção da Fatia A: .txt do Revit em português deve forçar tabulação no PapaParse; CSV salvo pelo Excel em português vem em Windows-1252; instalar o SheetJS pelo CDN oficial, porque o pacote xlsx do npm está desatualizado e tem falhas de segurança.

Vercel

O que é	Plataforma de hospedagem de sites e funções serverless.
Papel aqui	Hospeda o backend FastAPI (já no ar) e o site. O que a banca acessa é o link público.
Conecta com	GitHub (deploy do código), variáveis de ambiente configuradas na conta, Supabase e serviços externos pelo backend.
Configuração	Variáveis de ambiente sem aspas. Tempo máximo da função de pelo menos 60 s. Limite de 4,5 MB por envio. Não há vercel.json no repositório: a configuração está na conta.
Endereço	https://converge-challenge.vercel.app (/health e /docs)

P: Por que a pesquisa roda dentro da mesma requisição do POST /orcamentos?

A função serverless é encerrada quando a resposta é enviada, então qualquer trabalho em segundo plano depois dela seria interrompido. O backend pesquisa e grava item a item durante a requisição.

P: E se a pesquisa passar do tempo da função?
A função é encerrada no meio. Por isso o orçamento 'pesquisando' há mais de 3 minutos volta a poder ser reservado, e cada item já gravado permanece gravado.

P: Por que tudo na Vercel e não no notebook?
Para a demo não depender de uma máquina local: os jurados testam o link público.

P: Qual o limite de 4,5 MB?
Tamanho máximo de envio para a função. É o motivo de a importação de IFC (arquivos de 20 a 200 MB) ter sido planejada para ler o arquivo no navegador.

Docker e docker-compose

O que são	Docker empacota a API com suas dependências para rodar igual em qualquer máquina. docker-compose descreve os serviços a subir juntos.
Papel aqui	Ambiente de desenvolvimento e plano B: docker compose up --build sobe a API na porta 8000 com recarga automática. Perfil opcional n8n (porta 5678) e perfil local-db (Postgres local, que aplica os arquivos de sql/).
Conecta com	O mesmo .env do projeto (env_file). Supabase e Langfuse ficam na nuvem de propósito.
Código	Dockerfile, docker-compose.yml, .dockerignore

P: A produção usa Docker?
Não. O backend roda na Vercel. O Docker é para desenvolvimento e como plano B.

P: Por que Supabase e Langfuse não sobem localmente?
Consumem muita banda e memória, e a rede do evento é compartilhada. Ficam nos planos gratuitos na nuvem.

P: Para que serve o perfil local-db?
Um Postgres local como alternativa offline; os arquivos de sql/ são aplicados na criação.

n8n

O que é	Ferramenta de automação com fluxos visuais, usada para ligar canais (WhatsApp, e-mail, Slack) a serviços.
Papel aqui	O fluxo n8n/agent-bridge.json recebe um webhook e chama POST /agent/run. **Está fora do produto**: a decisão do projeto foi o site falar direto com o FastAPI.
Como subir	docker compose --profile n8n up --build; importar e publicar o fluxo uma vez (comandos no docker-compose.yml).

P: Por que o n8n está no repositório se não é usado no produto?
Foi a ponte inicial entre um canal e a API. Ficou como opção para integrar canais externos sem mexer no backend.

P: O n8n chama o agente de orçamento?
Chama o endpoint genérico do agente (/agent/run). O fluxo de orçamento passa por /orcamentos e pelo Realtime.

Testes: pytest e geradores de arquivos

O que são	pytest executa os testes. openpyxl, xlwt e msoffcrypto-tool geram arquivos de teste (XLSX, XLS legado e XLSX com senha).
Papel aqui	tests/importacao: 100 casos de leitura de arquivos, com contrato.py. tests/fatia_b: etapas 1 a 9 da pesquisa (configuração, busca, imagem, extração, cálculo, cache, pesquisa, orçamento no banco, interface, aquecimento). Marcador 'real' agrupa os testes que usam serviços de verdade.
Comando	python -m pytest tests -q (usar o Python do .venv)

P: Os testes custam dinheiro?
Os que usam rede e modelo, sim: gastam crédito da Featherless e cota do Tavily. Parte dos testes da etapa 6 executa a pesquisa de verdade. Vale rodar um arquivo por vez quando só uma área mudou.

P: Para que serve o marcador 'real'?
Separa os testes que dependem de serviços externos dos testes puramente locais, para poder rodar só os últimos.

P: Por que há geradores de arquivos?
Para criar formatos difíceis (XLS legado, XLSX com senha) que os testes de importação precisam.

Dados de teste sintéticos

O que são	docs/arquivos-teste: 183 arquivos de 12 obras fictícias em Recife e região (endereços e CEPs reais), em 15 formatos.
Conteúdo	Revit .txt, orçamento .xlsx, lista .csv, lista de WhatsApp .txt, SketchUp .csv e .html, memorial .docx, pedido de cotação e prancha em .pdf, foto de lista à mão (.jpg e .png), .xls, .ods, .json, .xml, .tsv, .md. Cerca de 116 produtos de marcas reais, escritos de 3 jeitos: completo, abreviado e genérico (que deve virar pendente). Arquivos com defeito de propósito: corrompido, vazio, colunas trocadas, PDF escaneado, duplicados, acentos quebrados, fórmulas com #REF!.
Índice	docs/arquivos-teste/LEIAME.md

P: Por que foram criados antes do produto?
Para os testes não serem feitos sob medida para o código dar certo.

P: São dados reais de clientes?
Não. Regra do repositório: só dados sintéticos. Os endereços e CEPs são reais para o cálculo de frete fazer sentido; as obras e os clientes são fictícios.

P: Para que servem na demo?
O botão "Usar arquivo de exemplo" usa 3 dessas obras, porque os jurados não têm um arquivo do Revit à mão.

Scripts de verificação

check_setup	python -m scripts.check_setup testa Featherless (chat e tool calling), Langfuse e Supabase. Gasta poucos centavos (duas chamadas curtas). Aceita --model para testar outro modelo.
aquecer_cache	python -m scripts.aquecer_cache pesquisa todos os itens da obra da demo e remove do cache os mais rápidos até sobrar a fração --manter (padrão 0,5; use 1 para a demo à prova de rede ruim).
sync_skills	bash scripts/sync_skills.sh gera as cópias das skills em .claude/skills e .agents/skills.

P: Quando rodar o check_setup?
Antes de concluir qualquer mudança e antes da demo. Se ele falhar no Langfuse com chaves certas, conferir se LANGFUSE_BASE_URL está definido.

P: O aquecer_cache muda o resultado da demo?
Muda a velocidade, não o conteúdo: os itens que ficam no cache aparecem na hora e os demais são pesquisados ao vivo.

Git, GitHub e arquivos de regras para assistentes

Git e GitHub	Repositório privado. Commits pequenos e frequentes, porque os organizadores podem pedir o histórico como evidência. Nunca versionar .env nem chaves.
AGENTS.md e CLAUDE.md	Regras do repositório lidas por assistentes de código (Claude Code, Codex, Cursor): estrutura de pastas, configuração só por config.py, toda ferramenta nova com argumentos Pydantic, decisões sobre pessoas sempre escaladas a humano, só dados sintéticos.
Skills (skills/*.md)	Guias no padrão Agent Skills: 01 agente ReAct, 02 integração com a Featherless, 03 tracing com Langfuse, 04 design da interface. A fonte única é skills/; as cópias em .claude/skills e .agents/skills são geradas.

P: Por que commits pequenos e frequentes?
Regra do evento: o histórico pode ser pedido como evidência de que o código foi feito durante o hackathon, a partir de sábado às 10h.

P: O que são as skills?
Instruções que o assistente de código lê antes de mexer em uma área (por exemplo, o padrão do agente ou a identidade visual). Mantêm o código dos integrantes consistente.

P: Por que há cópias das skills em outras pastas?
Claude Code e Codex procuram skills em pastas diferentes. As cópias são geradas por script a partir de uma fonte única para não divergirem.

Google Forms (Apps Script)

O que é	Pesquisa de validação com arquitetos criada por um script do Google Apps Script.
Papel aqui	forms/criar_pesquisa.gs cria o formulário "Pesquisa Rápida: Automação em Arquitetura" (momento profissional, tempo gasto em orçamentos, maior frustração do processo). Gera dados de validação de mercado para o pitch.
Como usar	script.google.com, novo projeto, colar o código e executar criarForm; os links de edição e de resposta aparecem nos registros.

P: Para que serve no pitch?
Metade da nota é negócio. As respostas dão números próprios sobre o tempo gasto e as dores do processo, em vez de estimativas sem fonte.

P: Esses números já existem?
Só depois de coletar respostas. Não citar percentuais antes de ter os dados.

5. O registro de um item: quem escreve e quem lê

O contrato entre as fatias é a linha de orcamento_itens. Os nomes abaixo vêm de src/modelos.py (campos que a Fatia B grava) e dos combinados com as Fatias A e C. O arquivo SQL único é da Fatia A e não está neste repositório.

Campo	Quem escreve	Quem lê	Observação
id, orcamento_id, descricao, quantidade, unidade, categoria, classificacao, linha	Fatia A (no POST /orcamentos)	Fatia B (colunas lidas por orcamento.py)	Item nasce com status 'buscando'. classificacao (definido ou pendente) vem da função classificar_item() da Fatia A
status	A (buscando); B (resultado)	Site, C	buscando, definido, pendente_especificacao, nao_encontrado, erro
produto_normalizado, imagem_url	B	Site	A imagem vem do cache quando o item já foi pesquisado
embalagem, qtd_embalagens	B	Site, C	qtd_embalagens já inclui a perda; arredondada para cima
preco_emb_min, preco_emb_med, preco_emb_max	B	Site, C	Preço **por embalagem** (não por m² nem por unidade de medida)
preco_un_min/med/max, unidade_preco	B	Site	Preço por unidade base (por exemplo R$/m²) quando a unidade permite
confianca, estimativa	B	Site, C	estimativa verdadeiro = faixa estimada de item genérico
fontes (lista)	B	Site, C	{loja, dominio, url, preco_embalagem, embalagem}. preco_embalagem é número maior que zero. dominio sem www (por exemplo leroymerlin.com.br)
premissa	B	Site	Conta usada, por exemplo "rendimento ≈ 8 m²/L"; exige coluna nova no SQL
pergunta, motivo	B	Site	pergunta: só em pendentes. motivo: pendentes, não encontrados e erros
preco_data, iteracoes, atualizado_em	B	Site	preco_data: quando o preço foi obtido (cache vencido mostra a data). iteracoes: quantas vezes o agente refez a busca
similar	ninguém ainda	C (sugestão "trocar por similar")	Não é gerado hoje; sem ele a sugestão não aparece

Campos de orcamentos

Campo	Quem escreve	Para quê
status_pesquisa (pendente, pesquisando, concluida, falhou), pesquisa_erro	B	Controla a reserva e libera novo clique em caso de falha
pesquisa_iniciada_em, pesquisa_concluida_em	B	Tempo real da pesquisa, usado no ganho
custo_usd	B	Soma do custo das chamadas ao modelo; gravado ao concluir e somado a cada repesquisa (exige coluna nova)
logistica, viabilidade, ganho	C	Resultado final mostrado na tela

6. Configuração por ferramenta

Tudo é lido de src/config.py e do .env. Nunca versionar o .env. Na Vercel, as variáveis não levam aspas.

Variável	Padrão	Ferramenta / efeito
FEATHERLESS_API_KEY	(chave)	Featherless: autenticação
FEATHERLESS_BASE_URL	https://api.featherless.ai/v1	Featherless: endereço da API
FEATHERLESS_MODEL	zai-org/GLM-5.2	Modelo do agente
FEATHERLESS_MODEL_FAST / _FALLBACK	(vazio) / Qwen/Qwen3-32B	Modelo barato / reserva em 400 e 503 (valores do README)
LLM_TEMPERATURE, LLM_TIMEOUT_SECONDS, LLM_MAX_RETRIES	0,2 / 60 / 3	Ajustes das chamadas ao modelo
AGENT_MAX_STEPS	8	Limite de passos do agente genérico
SUPABASE_URL, SUPABASE_KEY	(projeto, service_role)	Supabase: acesso do backend
LANGFUSE_ENABLED	true	Interruptor do tracing
LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY	pk-lf-..., sk-lf-...	Langfuse: autenticação
LANGFUSE_BASE_URL	https://us.cloud.langfuse.com	Langfuse: região US (LANGFUSE_HOST é o nome antigo)
TAVILY_API_KEYS	(chaves separadas por vírgula)	Tavily: rodízio entre contas
PESQUISA_MODELO_EXTRACAO	Qwen/Qwen3-32B	Modelo que extrai preços
PESQUISA_MODELO_AGENTE	(vazio = FEATHERLESS_MODEL)	Modelo do agente da pesquisa
PESQUISA_CONCORRENCIA_LLM / _BUSCA / _AGENTE	3 / 8 / 1	Chamadas simultâneas: extração, busca, agente
PESQUISA_AGENTES_SIMULTANEOS	2	Itens com agente ao mesmo tempo
PESQUISA_MIN_LOJAS	2	Abaixo disso o agente refaz a busca
PESQUISA_MAX_REFAZER	2	Buscas extras do agente por item
PESQUISA_TIMEOUT_AGENTE_S / _PRAZO_AGENTES_S	40 / 60	Tempo por item / prazo total da fase 2
PESQUISA_MAX_LOJAS	5	Limite de lojas consideradas por consulta
PESQUISA_CACHE_DIAS	7	Validade do cache de preços

7. Perguntas que podem aparecer

Sobre o agente e a IA

P: Como o agente funciona, em uma frase?

Pesquisa o preço, confere se é o mesmo produto, refaz a busca sozinho quando não acha lojas suficientes, pergunta ao arquiteto quando falta informação e entrega faixa de preço com fonte e confiança.

P: Onde está a IA e onde não está?

Na leitura dos anúncios (Qwen3-32B) e na reescrita da busca (GLM-5.2). Cálculo de preço, embalagens, perda, frete e viabilidade são código determinístico.

P: Como evitam preço inventado?

O preço só existe se estiver escrito no anúncio, só entra na média se o anúncio for do mesmo produto, e o modelo de dados recusa preço sem fonte.

P: Por que o sistema pergunta em vez de escolher?

Marca, linha e acabamento mudam o preço; a decisão é do arquiteto. O sistema mostra uma faixa estimada e faz uma pergunta objetiva.

P: Como a IA é auditada?

Pelo Langfuse: cada orçamento é uma árvore com um galho por item, mostrando consulta, resposta das lojas, raciocínio, tokens e custo.

P: Por que não usar um modelo único e maior?

Mais lento e mais caro nas tarefas simples. A divisão por tarefa foi medida: o Qwen3-32B extrai preço em ≈5 s; o GLM-5.2 só roda quando necessário.

Sobre dados e preços

P: De onde vêm os preços?

De anúncios das 18 lojas da lista fixa, encontrados pelo Tavily (ou DuckDuckGo, na reserva).

P: São preços reais de compra?

São preços de anúncio no momento da busca, não preços negociados. A tela mostra faixa, confiança e o link para o arquiteto conferir e negociar.

P: Como calculam as quantidades?

Partem da quantidade do item, aplicam a perda do tipo de material e dividem pelo conteúdo da embalagem escolhida, arredondando para cima. Algumas conversões usam premissas (por exemplo, rendimento de tinta), marcadas com "≈" na tela.

P: O que acontece se uma loja não tiver o produto?

Ela não entra na média. Com menos de 2 lojas, o agente tenta reformular a busca; se mesmo assim não houver, o item mostra a faixa com baixa confiança ou "não encontrado".

P: Os dados são reais?

Não. Usamos apenas dados sintéticos: 12 obras fictícias com endereços e CEPs reais de Recife e cerca de 116 produtos de marcas reais.

Sobre arquitetura e operação

P: Por que a pesquisa roda dentro da requisição?

A Vercel encerra a função quando a resposta é enviada, então qualquer trabalho depois dela seria interrompido. O backend grava item a item durante a requisição e o Realtime leva cada gravação à tela.

P: Por que o n8n está no repositório se não é usado?

Foi a ponte inicial Webhook → API. A decisão do projeto foi tirá-lo do produto; o arquivo fica como opção para canais como WhatsApp e e-mail.

P: O que acontece se o Langfuse ou o Tavily caírem durante a demo?

Sem o Langfuse, o produto continua igual, só sem traces. Sem o Tavily, entra o DuckDuckGo. Para a demo, o cache pré-aquecido evita depender da rede em metade dos itens.

P: É seguro?

Dados sintéticos; chave service_role apenas no backend; chaves fora do Git. Limites conhecidos: CORS aberto e rotas sem autenticação, aceitos para a demo.

P: Como testaram?

Testes automatizados (tests/fatia_b para a pesquisa, tests/importacao com 100 casos para a leitura de arquivos) e 183 arquivos realistas em 15 formatos, criados antes do produto.

Sobre negócio (ligação com o produto)

P: Qual é o ganho para o usuário?

A tela mostra o tempo estimado à mão contra o tempo real e o custo de IA daquele orçamento. O tempo à mão é uma estimativa (minutos por item × itens); o tempo real e o custo são medidos.

P: Qual o custo por orçamento?

É somado pelo sistema a cada chamada ao modelo. Citar apenas o valor medido nos testes, nunca uma estimativa.

P: Qual é o próximo passo do produto?

Comparação com o SINAPI (tabela oficial de custos), memória de marcas preferidas por escritório, integração com sistemas de gestão de obra, importação de IFC e pedidos de cotação prontos por loja. Hoje esses itens são plano, não funcionalidade.

8. O que não fazemos e o que está pendente

Dizer com clareza na banca

- Não negociamos com lojas, não fechamos compra e não temos preço de contrato.
- Não lemos IFC (formato BIM) ainda. A ideia está em docs/ideias/IFC-importacao.md: a leitura seria no navegador com web-ifc, porque arquivos reais têm 20 a 200 MB e a Vercel aceita 4,5 MB por envio.
- Comparação com o SINAPI e memória do escritório são apenas parte do pitch.
- Os preços são de anúncios, não de pedidos.
- Rotas sem autenticação e CORS aberto (demo).
- Não há números de acerto, custo médio ou tempo médio por orçamento publicados: ainda precisam ser medidos rodando os arquivos de teste (tarefa da Fatia D).

Pendências de integração

Pendência	De quem	Efeito se não for resolvida
Juntar a branch feat/joao-complementos-e2 (src/rotas_resultado.py) e incluir o router no main.py	João e Samuel	Sem frete, viabilidade e ganho: o gancho final é pulado
Colunas custo_usd (em orcamentos) e premissa (em orcamento_itens) no SQL único	Samuel	Custo de IA some da tela; gravação de premissa pode falhar
Cadastrar fornecedores com os mesmos domínios da lista de lojas	João	Itens caem em "sem loja" no frete
Gerar o campo similar nos itens (opcional)	Fatia B	Sugestão "trocar por similar" não aparece
Medir métricas para o pitch (itens resolvidos sozinhos, % que precisaram refazer, tempo e custo médios)	Fatias B e D	Pitch sem números próprios
Variáveis de ambiente da Vercel: LANGFUSE_BASE_URL (US), sem aspas; tempo máximo da função ≥ 60 s	Time	Traces podem falhar; pesquisa pode ser cortada

9. Cola rápida

Item	Valor
Modelo do agente	zai-org/GLM-5.2
Modelo de extração (reserva)	Qwen/Qwen3-32B (google/gemma-4-31B-it)
Busca	Tavily (principal), DuckDuckGo (reserva); 18 domínios fixos
Cache	7 dias; chave = descrição normalizada
Mínimo de lojas	2 (abaixo disso o agente refaz a busca, até 2 vezes)
Tempo do agente	40 s por item; 60 s de prazo total da fase 2
Outliers	fora de 40% a 250% da mediana
Confiança	alta: ≥3 lojas e dispersão ≤25%; média: ≥2 lojas e ≤50%; baixa: o restante
Concorrência	8 buscas, 3 extrações, 1 chamada do agente, 2 itens com agente
Status do item	buscando, definido, pendente_especificacao, nao_encontrado, erro
Status do orçamento (pesquisa)	pendente, pesquisando, concluida, falhou
Status do agente genérico	completed, escalated, max_steps_reached, error
Langfuse	região US; um trace por orçamento, um galho por item
Backend em produção	https://converge-challenge.vercel.app (/health e /docs)
Comando de verificação	python -m scripts.check_setup
Subir local	uvicorn src.main:app --reload (porta 8000)

10. As fatias do time

O trabalho foi dividido em quatro **fatias**, cada uma com um dono e com arquivos próprios, para que ninguém mexa nos mesmos arquivos ao mesmo tempo. As fatias conversam por **contratos** (funções e campos do banco combinados antes). Esta seção explica o que é cada uma e o que ela faz; a descrição vem de docs/FATIAS-DO-TIME.md e do código deste repositório (que contém a Fatia B).

Fatia	Dono	Nome	Em uma linha
A	Samuel	Entrada e revisão	Do upload do arquivo até "Enviar para a IA": tela, tabela editável, revisor de erros, API de entrada e arquivo SQL único
B	Enzo	Pesquisa agêntica de preço	Para cada item: pesquisa, confere, refaz, calcula embalagens, grava e atualiza a tela ao vivo
C	João	Itens esquecidos, frete e resultado	Complementos, frete por CEP, viabilidade, ganho de tempo e exportação
D	a definir	Negócio e apresentação	Mercado, pitch, vídeo, submissão e métricas; vale metade da nota

Funções do produto e a fatia de cada uma

#	Função	O que faz	Fatia
E1	Revisor de erros	Aponta e sugere correção com um clique: unidade incompatível, quantidade absurda, duplicados com nomes diferentes, colunas trocadas	A
E2	Itens esquecidos	Sugere itens que costumam faltar (porcelanato pede argamassa e rejunte; pintura pede selador e massa) com a quantidade calculada	C (mostrado pela A)
E3	Lista de compra com perda	"180 m² de tinta" vira "2 latas de 18 L"; porcelanato com 10% de perda vira número de caixas	B
E4	Pesquisa agêntica	Pesquisa, confere se é o mesmo produto e refaz a busca sozinha, até 2 vezes	B
E5	Pergunta ao arquiteto	Item genérico vira pergunta objetiva; ao responder, é pesquisado de novo	B
E6	Confiança e fonte	Todo preço tem link da loja; confiança alta, média ou baixa. Nunca preço sem fonte	B
E7	Frete por CEP	Loja única ou dividir entre lojas, com frete total e economia	C
E8	Viabilidade	Compara o total com o orçamento do cliente: cabe ou falta R$ X, com 3 sugestões de economia	C
E9	Arquivo de exemplo	Botão "Usar arquivo de exemplo" com 3 obras de docs/arquivos-teste	A
E10	Ganho na tela	Tempo à mão estimado contra o tempo real, mais o custo de IA	C (com dados da B)
E11	Exportar	Plano de compra em Excel ou PDF, agrupado por loja	C
T1	Pedidos de cotação (se der tempo)	Pedido por loja pronto para enviar, com texto de negociação	C
T2	Similar mais barato (se der tempo)	Sugestão de produto equivalente mais barato, sempre como sugestão	B
T3	Progresso ao vivo e "por quê?" (se der tempo)	"12 de 20 itens" e o raciocínio de cada item	B

Fatia A: Entrada e revisão (Samuel)

O que é	A porta de entrada do produto: tudo o que acontece antes de a IA pesquisar e a estrutura da página de resultado.
Tela	Nova obra (nome, CEP e orçamento do cliente em R$); upload de CSV, Excel, TXT do Revit e PDF com texto; botão "Usar arquivo de exemplo"; tabela editável com os avisos do revisor de erros e as sugestões de itens esquecidos, cada uma com botão "Aceitar"; botão "Enviar para a IA"; esqueleto da página de resultado e o hook useOrcamento(id), que escuta o Realtime das duas tabelas.
Backend	POST /orcamentos valida o contrato (tests/importacao/contrato.py), grava orcamentos e orcamento_itens com status 'buscando' e chama pesquisar_orcamento(id) da Fatia B na mesma requisição. classificar_item() decide definido ou pendente antes da busca (função única; a B confirma e a C só lê). revisar_itens() aplica as regras do E1 em Python; o modelo só redige a sugestão.
Dados	Dono do arquivo SQL único (orcamentos, orcamento_itens, cache_precos, fornecedores), juntando os campos que B e C pedirem. Roda os 100 casos de tests/importacao e os arquivos de docs/arquivos-teste; arquivo ruim deve gerar mensagem amigável.
Cuidados	.txt do Revit em português deve forçar tabulação no PapaParse; CSV do Excel vem em Windows-1252; instalar o SheetJS pelo CDN oficial (o pacote xlsx do npm tem falhas de segurança).
Neste repositório	Não está aqui, exceto o contrato de entrada em tests/importacao/contrato.py.

P: O que a Fatia A entrega para a B?

Itens já revisados pelo arquiteto, gravados em orcamento_itens com status 'buscando', e a chamada pesquisar_orcamento(orcamento_id) na mesma requisição.

P: Por que a revisão acontece antes da pesquisa?

Corrigir uma unidade errada ou um duplicado antes evita gastar buscas e crédito de modelo com itens que o arquiteto ia corrigir de qualquer forma.

P: Quem decide se um item é genérico?

classificar_item(), da Fatia A. A Fatia B confirma durante a pesquisa e a C apenas lê o resultado.

P: Por que o dono do SQL é a Fatia A?

Um arquivo único evita três versões conflitantes das mesmas tabelas. As colunas que B e C precisam são pedidas a quem mantém o arquivo.

Fatia B: Pesquisa agêntica de preço (Enzo)

O que é	O coração do produto: transforma cada item da lista em preço com fonte, número de embalagens e nível de confiança, gravando ao vivo. É a fatia deste repositório.
Backend	pesquisar_orcamento(id) e repesquisar_item(item_id) em src/orcamento.py; busca (src/busca.py); extração e conferência (src/precos.py); agente que refaz a busca (src/pesquisa.py); cálculo (src/calculo.py); lista de compra com perda (src/compra.py); cache (src/cache.py); custo (src/custo.py). Rota POST /orcamento-itens/{id}/repesquisar em src/main.py.
Por item	cache, busca, extração, agente se faltarem lojas, cálculo, lista de compra, confiança e fontes, gravação. Item genérico vira faixa estimada e pergunta. Falha total: cache vencido com data, ou "não encontrado" com botão Pesquisar de novo.
Tela	ui-fatia-b/: ItemRow, cota de preço (PriceDimension), StatusBadge e prévia com Realtime simulado. A Fatia A integra esses componentes à página de resultado.
Demo	scripts/aquecer_cache.py; checar a chave anon e as regras de leitura (RLS) no Supabase e o tempo máximo da função na Vercel (60 s ou mais).
Langfuse	Uma árvore por orçamento e um galho por item, com as iterações visíveis para o pitch.
Entrega para a C	Por item: fontes (loja, dominio, url, preco_embalagem), embalagem, qtd_embalagens, preco_emb_min/med/max, confianca, estimativa. No fim, chama src.rotas_resultado.resultado(orcamento_id).

P: O que a Fatia B garante?

Que nenhum preço aparece sem fonte, que o preço vem do mesmo produto pedido, que a quantidade já inclui a perda e que cada item é gravado assim que fica pronto.

P: O que a Fatia B não decide?

Marca e acabamento de item genérico (pergunta ao arquiteto), e qualquer decisão de compra: o sistema sugere, o arquiteto decide.

P: Quais partes dependem de outras fatias?

A leitura dos itens e a classificação vêm da A; o gancho final chama a função da C; as colunas premissa e custo_usd dependem do SQL do Samuel.

P: O que ainda não foi feito na B?

O campo similar (sugestão "trocar por similar", T2) não é gerado hoje, e faltam as métricas de acerto, tempo e custo médios para o pitch.

Fatia C: Itens esquecidos, frete e resultado (João)

O que é	Tudo o que acontece depois que os preços existem: o que falta comprar, onde comprar, se cabe no orçamento e quanto tempo e dinheiro foram poupados.
Estado	Está na branch feat/joao-complementos-e2, ainda sem push; será juntada com as demais. Esta descrição é o combinado, não código deste repositório.
Complementos (E2)	sugerir_complementos(itens): regras por categoria (porcelanato pede argamassa, rejunte e nivelador; pintura pede selador, massa e fita ou lixa; vaso pede válvula e assento; drywall pede perfis, massa, fita e parafuso), com a quantidade calculada. A Fatia A mostra na revisão.
Frete (E7)	Consulta o CEP da obra (ViaCEP ou BrasilAPI) e cruza com fornecedores (CEP, frete base, R$ por km, pedido mínimo). Resposta: estratégia loja_unica ou dividir, lojas, frete total e economia estimada. Usa os preços por loja de fontes[], casando fontes[].dominio com fornecedores.
Viabilidade (E8)	Total mínimo, médio e máximo contra o orçamento do cliente: cabe ou falta R$ X, com 3 sugestões de economia (trocar por similar, dividir frete, revisar itens pendentes). Itens nao_encontrado, erro e buscando ficam fora do total e a tela avisa.
Ganho (E10)	Tempo à mão estimado (minutos por item × itens) contra o tempo real (pesquisa_iniciada_em e pesquisa_concluida_em), mais o custo de IA (custo_usd).
Exportar (E11)	Plano de compra em Excel ou PDF agrupado por loja. (T1, se der tempo: pedidos de cotação por loja com texto de negociação.)
Dados	Cadastrar os fornecedores no Supabase com as mesmas redes que a busca encontra (Leroy Merlin, C&C, Telhanorte e outras, com lojas de Recife), com dominio, CEP, frete base, R$ por km e pedido mínimo.
Como a B chama	src.rotas_resultado.resultado(orcamento_id), uma chamada que faz frete, viabilidade e ganho e grava em orcamentos (logistica, viabilidade, ganho, concluido_em). É seguro chamar de novo; também existe POST /orcamentos/{id}/resultado.

P: O frete e a viabilidade usam IA?

Não. São funções Python que recebem dados e devolvem números, como o cálculo de preço.

P: De onde vêm os valores de frete?

Da tabela fornecedores cadastrada pelo time. O preço do produto vem da busca; o frete vem do cadastro. Confirmar com o João a origem exata dos valores antes de afirmar algo no pitch.

P: Por que a chave de ligação é o domínio?

fontes[].dominio (por exemplo leroymerlin.com.br) é o que casa o preço encontrado com a loja cadastrada. Se o domínio não bater, o item cai em "sem loja" no frete.

P: O que acontece se a branch da C ainda não foi juntada?

O gancho da Fatia B só registra no log que o módulo não existe e pula; a pesquisa funciona, só não há frete, viabilidade nem ganho.

Fatia D: Negócio e apresentação

O que é	Tudo o que transforma o produto em nota: viabilidade como startup e pitch valem metade da avaliação. O doc do time deixa o dono a definir; confirmar quem assumiu.
Mercado	Quantos arquitetos e escritórios existem (fonte: CAU/BR), quanto tempo leva um orçamento à mão (mentores, arquitetos e a pesquisa do Google Forms), quanto cobrar (por orçamento ou assinatura por escritório).
Por que agora	IA que pesquisa e confere sozinha ficou barata; o custo real por orçamento é medido pelo próprio sistema.
Entregas	Slides, roteiro, vídeo (domingo de manhã) e submissão até 12h30 de domingo (prazo final 13h).
Prêmio AWS	Domínio .com.br, landing simples e e-mail no mesmo domínio (Gmail não vale), mais conta AWS.
Métricas	Com a Fatia B: rodar os arquivos de docs/arquivos-teste e medir % de itens resolvidos sozinhos, % que precisaram refazer a busca, tempo e custo médios por orçamento.
Só no pitch	Comparação com o SINAPI, memória de marcas por escritório e integração com sistemas de gestão de obra: citados como próximos passos, não codificados.

P: Como a banca avalia?

Quatro critérios de peso igual: protótipo e aderência (os jurados testam o link), viabilidade como startup, pitch e criatividade. A Canastra e a Tako estão na banca.

P: Como o pitch usa o sistema?

Em 3 minutos: a dor com número, a solução em uma frase, a demo ao vivo (cerca de 30 s rodando), o negócio e o ganho contra o custo por orçamento.

P: Que números dependem da Fatia B?

Itens resolvidos sozinhos, itens que precisaram refazer a busca, tempo médio e custo médio por orçamento. Só podem ser citados depois de medidos.

Contratos entre as fatias

De → Para	Contrato
A → B	pesquisar_orcamento(orcamento_id), chamada na mesma requisição; os itens começam com status 'buscando'
A ↔ C	POST /revisao/sugestoes devolve {erros (A), complementos (C)}, mostrados na tabela de revisão
B → C	Por item: fontes [{loja, dominio, url, preco_embalagem}], embalagem, qtd_embalagens, preco_emb_min/med/max, confianca, estimativa
B → C (gatilho)	No fim, a B chama src.rotas_resultado.resultado(id), que grava em orcamentos; o Realtime atualiza o resultado
Todos	Status do item: buscando, definido, pendente_especificacao, nao_encontrado, erro